
Testing the performance of ResNet and its variants as feature extractors in Few-shot learning

Yiming(Eric) Cao
Student ID: 92138098
ericcym@ece.ubc.ca

Abstract

Traditional machine learning approaches rely on large labeled datasets to train models for specific tasks to achieve high performance. However, collecting and labeling new data is expensive and impractical in many real-world scenarios. Few-shot learning enables models to learn new concepts quickly and accurately from a few examples. Usually, CNNs are utilized as feature extractors in Few-shot learning since they are designed to capture spatial information in an image. But there are many choices for CNN. In this project, We will evaluate the most widely used feature extractor in few-shot learning: ResNet and two variants of it: ConvNeXt and DenseNet, on the datasets CUB. In consequence, ResNet exhibited exceptional performance on classification accuracy and training time. DenseNet marginally outperformed ResNet after data enhancement, and it has a strong generalization but suffers from long training time. ConvNeXt, exhibited poor performance and had a longer training time due to its unsuitability for data sets with limited data.

1 Introduction

Traditional machine learning approaches rely on large labeled datasets to train models for specific tasks to achieve high performance. However, collecting and labeling new data is expensive and impractical in many real-world scenarios. In contrast, humans can learn new concepts quickly with limited information by leveraging their past experiences as a reference. For instance, if someone has never seen a dog before but has seen many other animals, they can quickly distinguish a dog by looking at a few pictures. To simulate this human learning ability, Few-shot learning is born and has recently garnered considerable attention. It enables models to learn new concepts quickly and accurately from few examples(1), reducing learning costs and time compared to conventional machine learning.

Similar to Few-shot learning, transfer learning also aims to improve learning on a new task by leveraging existing knowledge, so the existing transfer learning knowledge can be referred to in the study of Few-shot learning. Transfer learning utilizes a pre-trained model as a backbone for a new task. This pre-trained model acquires features from the related tasks, and transfers to the new task. Similarly, in Few-shot learning, a pre-trained model is employed as a feature extractor, exploiting knowledge from related tasks and making predictions on new tasks by this knowledge. However, unlike transfer learning, Few-shot learning emphasizes the capacity to learn novel concepts from a limited dataset. Consequently, in Few-shot learning, data augmentation and the accurate classification of feature representations assume greater significance.

Few-shot learning has broad applications across a range of domains, including natural language processing, object recognition, and image classification. In this project, we concentrate exclusively on few-shot image classification, which involves recognizing new categories with a limited number of labeled images. To achieve this, we utilize a specific training process comprising a support set and a query set for each training and testing set. The support set consists of K images that enable the model

to learn, called K-shot. The query set consists of Q images the model must accurately classify, called Q-query. During the training and testing, the network may not process all the data in the dataset. Instead, we typically sample E batches in the dataset, with replacement, called E-episodes, and each episode contains N classes that need to be classified, called N-ways. N-way K-shot is an essential setting for Few-shot learning under ordinary situations(2).

2 Related Work

There are three commonly used approaches exist to address the challenge of Few-shot learning, which are the optimization-based methods, the Metric-based methods, and the Metric-based enhanced methods.

2.1 Optimization-based

Optimization-based Few-shot learning methods do not rely on external metric indicators that depend on the model. Instead, they use model-agnostic methods, using stochastic gradient descent, to define a general optimization method compatible with all models. Applying this algorithm optimizes all potential classes rather than optimizing for a specific dataset. The most popular optimization-based method is the Model-Agnostic Meta-Learning (MAML) algorithm(3). MAML is a method that focuses on the model's initialization to find the optimal initialization parameter that can be quickly fine-tuned on new tasks. Reptile(4) is a variant of MAML that updates the parameters of the network model by taking the average of the gradients obtained on a few learned examples rather than directly using gradient descent. Optimization-based methods have the major drawback of being computationally expensive due to the presence of two nested loops in the training process, one for the structural parameter of the model and another for the initial parameter of the model.

2.2 Metric-based

A metric-based model considers the similarity of input data and clusters query sets based on the information from the support set. One of the most well-known examples is the Prototypical Networks(5). This model utilizes feature extractors (CNNs) to map both the support and query set to a feature space. It defines the average value of each class in the support set as the prototype feature. It employs the Euclidean distance between the query set features and the prototype feature to classify each query image. The Siamese network(6) proposes a similarity metric that measures pairs of images and produces a similarity score based on the distance between their feature representations. LaplacianShot (7) is based on a Siamese network architecture, where two identical networks share weights and extract features from the support and query images. And the Laplacian regularization term is used to smooth the feature space and improve the model's generalization ability. Therefore, choosing the distance metric to compute similarity is always the major challenge of metric-based methods.

2.3 Augmented metric-based

Augmented metric-based methods use data augmentation to generate additional data to improve the robustness and generalization of the model. S2M2(8) is a classical approach of Augmented metric-based, employing the Manifold mixup to mix the features of images from two different classes during training to learn a more generalizable feature representation. In the paper(9), the author proposes a method to alleviate the overfitting effect in few-shot learning from the perspective of data distribution estimation. It assumes that each class's prior distribution of features is a Gaussian distribution and then calibrates data by using the mean and variance of each class to obtain a more generalized model.

3 Method

In this project, the first step is to extend the dataset by data augmentation. After that, a CNN will be trained as a feature extractor. The Prototypical Networks will be used for classification at the final step.

3.1 Data augmentation

Data augmentation is a common technique used in computer vision tasks to expand a dataset's size by creating image transformations. There are some data augmentation methods applied in this project.

- Horizontal Flipping: Flip the image horizontally
- Random Cropping: Randomly crop a portion of the image
- Color Jittering: Randomly change the brightness, contrast, saturation, and hue of the image
- Center cropping: crop the center square of the image

Data augmentation is a beneficial technique for few-shot learning because it can increase the dataset's size and improve the robustness and generalization of models.

3.2 Feature extractor

In this project, CNNs will be utilized as feature extractors to produce image feature representations. CNNs are good feature extractors of images because they are designed to capture spatial information in an image. Typically, CNNs contain multiple convolutional layers that take in images or the output of the previous convolutional block as input and map them to a feature space of a specific dimension. We will evaluate the most widely used feature extractor in few-shot learning: ResNet(10) and two variants of it: ConvNeXt(11) and DenseNet(?), on the given datasets.

3.3 Classifier

In this project, Prototypical Network is used as a classifier, which is a classical metric-based method. It employs the Euclidean distance between the feature representations of the query set and the prototype to classify each query image. During the training process, the feature extractor mentioned earlier is used to extract features for both the support set and the query set. Prototypes for each class are then calculated by averaging all the features of the support set. Finally, the Euclidean distance between the query set features and the prototype is calculated to classify the query images. Here is the pseudo-code:

Algorithm 1 Prototypical classifier

Input: feature extractor: CNN, support set: K-Shot, query set: Q-Query, class number N

Output: Loss of 1 episode

```
images-K, labels-K = K-Shot
features-K = CNN(images-K)
images-Q, labels-Q = Q-Query
features-Q = CNN(images-Q)
prototypes = []
for L = 1,2,...,N do
    cluster index = index(labels-K[labels-K = L])
    cluster = features-K(cluster index)
    prototype-L = mean(cluster)
    prototypes.append(prototype-L)
end for
cluster distance = Euclidean distance(prototypes, features-Q)
probability-Q = Softmax(- cluster distance)
Loss = CrossEntropyLoss(probability-Q, labels-Q)
```

Further explanation about the pseudo-code. In the for loop, we can consider each class in N as a cluster, so the classical clustering algorithm like k-means can be used to calculate each prototype. However, training an additional model inside the Prototypical classifier is extremely expensive. Therefore, the means of features are directly used as the prototype, as suggested in the original paper. It's important to note that all "-" symbols in this pseudo-code refer to a hyphen, not a minus sign, except for the element inside the Softmax. The negative of the distance between image features and each prototype directly represents the probability of images belonging to each cluster. When

the distance between an image feature and a prototype is lower, the probability of belonging to that cluster is larger.

In the original, the main contribution of this report was to improve the original Prototypical Networks, but they all failed due to various situations, and the specific reasons were added to the appendix. Instead, I did experiments evaluating feature extractors and data augmentation.

4 Experiment

4.1 General settings

Before diving into the experiment part, it's necessary to first introduce some general settings. All of the code used in this project was implemented with Python, utilizing widely-used tools, e.g., Numpy, Pytorch, etc. The code was written in the form of a Jupiter notebook and run on a GPU in Colab.

Regarding the few-shot image classification, the general settings is 5-ways, 5-shot, 5-query, 500 episodes for the training set, and 50 episodes for the testing set. Specifically, 500 episodes were sampled from the training set, and 50 episodes were sampled from the testing set with replacement. Each episode consists of 5 classes (5-ways), and each class contains 10 images, 5 images belonging to the support set (5-shot) and 5 belonging to the query set (5-query).

Regarding the model architecture, we utilized the transform method provided by torchvision for data augmentation. For feature extraction, we employed the built-in models resnet18 and densenet121, and convnexttiny from torchvision.models without their fully-connect(classification) layers.

4.2 Train and test on CUB

CUB is a commonly used dataset in the field of computer vision. It consists of 11,788 images of 200 bird species, with each image labeled with the corresponding bird species.

To compare the performance of ResNet, DenseNet, and ConvNeXt feature extractors, we trained our model using these three CNNs with the same hyperparameters. We used the Adam optimizer with an learning rate of 0.001, the model was trained for a total of 100 epochs. Below are the results of our training

Table 1: Table for Classification accuracy and training time on CUB by using ResNet and DenseNet as feature extractor

	Accuracy (%)	Training time (hours)
ResNet	81.84 +- 1.2	1
DenseNet	83.76 +- 0.9	2
ConvNeXt	78.56 +- 1.5	2.5

Each CNN runs 10 times as a quantitative analysis, training time is for 100 epoch.

4.3 Discussion

Resnet is a widely-used feature extractor in few-shot learning and has been prominently featured in many outstanding research papers in recent years. In our experiments, Resnet also demonstrated exceptional performance. Its success may be attributed to the residual structure, which solves the vanishing or exploding gradients that occur in deep neural networks. This enables the deep network model acquire a powerful representation learning ability.

DenseNet, a variation of ResNet, has better resistance to overfitting. This is because traditional CNNs only rely on the output of the last layer, while DenseNet has connections between the outputs of each convolutional block. This allows the model to use detail features in low-complexity shallow layers and global features in high-complexity deep layers. This is particularly important when working with limited data. DenseNet performed similarly to ResNet in our experiments but with slightly higher accuracy. It achieved the highest correct rate among the three models we tested. Although DenseNet has fewer neurons (i.e., the number of channels in the convolutional layer) than ResNet, it runs slowly and takes more than twice the training time of ResNet. This seems counterintuitive since one would expect a network with fewer parameters to run faster. We hypothesize that a large feature

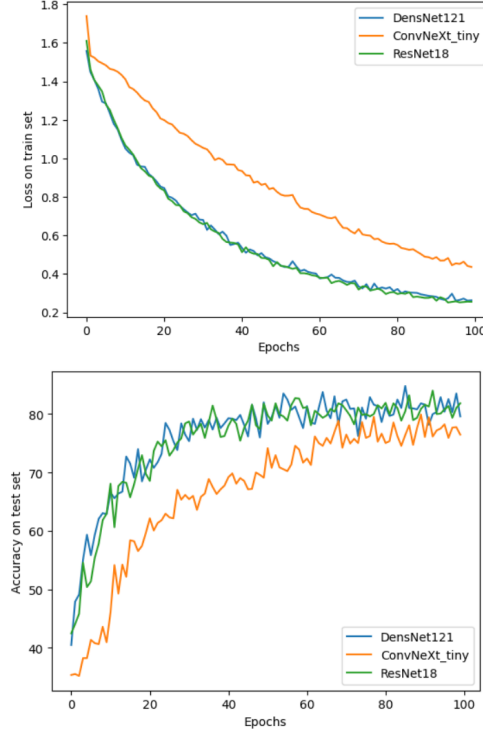


Figure 1: Training loss and testing accuracy curves

map which leads to a significant amount of calculation between the input and output of the hidden layers, resulting in a slow training time.

ConvNeXt, as a CNN proposed in recent years, has even outperformed ViT in some works. However, our experiments have shown that it does not perform well and is even worse than its predecessor, ResNet. We speculate that this may be due to ConvNeXt using a larger convolution kernel(7×7), which enlarges the receptive field and captures more global information but loses the crucial semantic information of the details. In few-shot learning, it restricts the features that the model can learn due to the small dataset. The prototypical network used in this project also relies on feature representations to classify, and the loss of semantic information may prove fatal in this context. Additionally, ConvNeXt uses an attention mechanism. The paper(12) suggests that the attention mechanism may contribute to the overfitting effect of image classification on small datasets.

4.4 Ablation study on data augmentation

Data augmentation is an effective technique to enhance the generalization and robustness of a model. However, inappropriate operations can lead to adverse effects, such as the use of a weak model or augmented data that falls outside the data domain. Therefore, in this part, we conducted an ablation experiment to investigate the effect of removing data augmentation.

Table 2: Table for Ablation study on data augmentation by using ResNet and DenseNet as feature extractor

	Accuracy (%)	decline (%)
ResNet	69.28	12.56
DenseNet	74.56	8.80

The model without data augmentation did not show a better result, indicating that the data augmentation method was successful. However, this ablation study is not entirely without value. After removing data augmentation, we observe that the accuracy of DenseNet does not drop as much as ResNet, demonstrating that DenseNet possesses a more robust generalization.

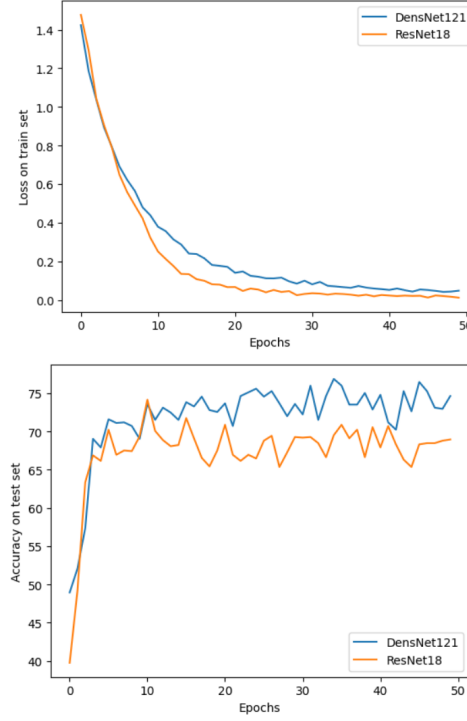


Figure 2: Training loss and testing accuracy curves without data augmentation

5 Conclusion

To summarize this report, we utilized three CNNs as feature extractors to address few-shot image classification. ResNet, the most classic feature extractor in this field, exhibited exceptional performance on classification accuracy and training time. DenseNet marginally outperformed ResNet after data enhancement, and it has a strong generalization but suffers from long training time. On the other hand, ConvNeXt, a relatively new architecture, exhibited poor performance and had a longer training time than the other two due to its unsuitability for data sets with limited data.

In future work, we intend to explore more types of CNNs to develop a unique feature extractor specifically designed for few-shot learning. Additionally, we discovered that data augmentation can significantly enhance model performance through ablation study. Therefore, we plan to incorporate more advanced data augmentation methods in few-shot learning in the future, such as using self-supervised learning to expand the dataset.

References

- [1] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, “Generalizing from a few examples: A survey on few-shot learning,” *ACM computing surveys (csur)*, vol. 53, no. 3, pp. 1–34, 2020.
- [2] Y. Bendou, Y. Hu, R. Lafargue, G. Lioi, B. Pasdeloup, S. Pateux, and V. Gripon, “Easy: Ensemble augmented-shot y-shaped learning: State-of-the-art few-shot classification with simple ingredients,” *arXiv preprint arXiv:2201.09699*, 2022.
- [3] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International conference on machine learning*, pp. 1126–1135, PMLR, 2017.
- [4] F. Corradini, M. Loreti, M. Piangerelli, and G. Rocchetti, “Reptile: A proactive real-time deep reinforcement learning self-adaptive framework,” *arXiv preprint arXiv:2203.14686*, 2022.
- [5] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” *Advances in neural information processing systems*, vol. 30, 2017.
- [6] G. Koch, R. Zemel, R. Salakhutdinov, *et al.*, “Siamese neural networks for one-shot image recognition,” in *ICML deep learning workshop*, vol. 2, Lille, 2015.
- [7] I. Ziko, J. Dolz, E. Granger, and I. B. Ayed, “Laplacian regularized few-shot learning,” in *International conference on machine learning*, pp. 11660–11670, PMLR, 2020.
- [8] P. Mangla, N. Kumari, A. Sinha, M. Singh, B. Krishnamurthy, and V. N. Balasubramanian, “Charting the right manifold: Manifold mixup for few-shot learning,” in *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pp. 2218–2227, 2020.
- [9] S. Yang, L. Liu, and M. Xu, “Free lunch for few-shot learning: Distribution calibration,” *arXiv preprint arXiv:2101.06395*, 2021.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- [11] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11976–11986, 2022.
- [12] H. Zhu, B. Chen, and C. Yang, “Understanding why vit trains badly on small datasets: An intuitive perspective,” *arXiv preprint arXiv:2302.03751*, 2023.

Some failed and unresolved ideas are included in the appendix

A Appendix - Prototypical Network with kernel function

When using the Prototypical Network, we treat each class in the feature map as a cluster in the few-shot image classification. This approach may result in a classical clustering problem where the clusters are not linearly separable. To address this issue, we can apply a kernel function to map the image features to a higher dimension. However, most feature extractors' final output feature dimensions are typically greater than 500, which is unlikely linear inseparability. Alternatively, if we can find a feature extractor that can learn features accurately and produce a tiny feature dimension, we can also solve this issue by adding a fully connected layer.

B Prototypical Network with attention mechanism

In the Prototypical Network, When computing the prototype for each cluster in the Prototypical Network, we take the mean of features in the cluster, assuming that all learned features have equal weight. However, this is incompatible with real-life scenarios, especially with small training sets where some noise features may be learned. To address this issue, we can introduce learnable weights by using the attention mechanism. The transformer architecture has a formula for attention, denoted as follows.

$$z = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Mapping the query Q, key K, and value V to the Prototypical Network, where the query is the features of the query set, the key is the prototype, and the value is the features of the support set, we can interpret this as the similarity between the prototype and the features in each query set as the weight of the features in the support set, which calibrates the distribution of each cluster. The pseudocode is as follows.

Algorithm 2 Prototypical classifier with attention mechanism

Input: feature extractor: CNN, support set: K-Shot, query set: Q-Query, class number N

Output: Loss of 1 episode

```

images-K, labels-K = K-Shot
features-K = CNN(images-K)
images-Q, labels-Q = Q-Query
features-Q = CNN(images-Q)
prototypes = []
for L = 1,2,...,N do
    cluster index = index(labels-K[labels-K = L])
    cluster = features-K(cluster index)
    prototype-L = mean(cluster)
    prototypes.append(prototype-L)

```

end for

```

similarity = dot product(prototypes, features-Q)
weight = Softmax(normalize(similarity))
new features-K = dot product(weight, features-K)
new prototypes = []

```

```

for L = 1,2,...,N do
    cluster index = index(labels-K[labels-K = L])
    cluster = new features-K(cluster index)
    prototype-L = mean(cluster)
    new prototypes.append(prototype-L)

```

end for

```

cluster distance = Euclidean distance(new prototypes, features-Q)
probability-Q = Softmax(- cluster distance)
Loss = CrossEntropyLoss(probability-Q, labels-Q)

```

However, the main challenge is that this idea cannot be directly applied to the original equation since the dimensions of query, key, and value are different in our case. Therefore, we need to develop a new formula to calculate the attention weight. This idea will be an intuition of our future direction.